A Tour of C++

- 확인 및 시험

목차

제1장. 기초 3
제2장. 사용자 정의 타입 4
제3장. 모듈화 5
제4장. 클래스 6
제5장. 필수적인 연산 7
제6장. 템플릿 8
제7장. 컨셉과 제네릭 프로그래밍 9
제8장. 라이브러리의 개요 10
제9장. 문자열과 정규 표현식 11
제10장. 입력과 출력 12
제11장. 컨테이너 13
제12장. 알고리즘 14
제13장. 유틸리티 15
제14장. 수치 16
제15장. 동시성 17
제16장. 역사와 호환성 18

제1장. 기초

핵심 정의

타입: **타입**은 **객체**가 취할 수 있는 값과 연산의 집합을 정의한다. [U01-001] (1.4, p.26)

객체: **객체**는 어떤 **타입**의 값을 갖는 메모리다. [U01-158] (1.4, p.26)

변수: **변수**는 명명된 **객체**다. [U01-160] (1.4, p.26)

상수의 원문 구분

구분	책의 설명
const [U01-170] (1.6, p.33)	**const**는 이 값을 바꾸지 않는다고 약속하며, **const**의 값은 실행 시간에 계산할 수 있다. [U01-163] (1.6, p.33)
constexpr [U01-171] (1.6, p.33)	**constexpr**는 이 값을 컴파일 시간에 평가한다는 의미이며, **constexpr**의 값은 반드시 컴파일 시간에 계산된다. [U01-002] (1.6, p.33)

제2장. 사용자 정의 타입

핵심 정의

생성자: 해당 클래스와 이름이 동일한 멤버 함수를 **생성자**라고 하며, 클래스를 **객체**화할 때 사용한다. [U02-075] (2.3, p.54)

union: **union**은 모든 멤버가 같은 메모리 주소에 할당되는 **struct**이며, 공용체의 크기는 가장 큰 멤버의 크기와 같다. [U02-078] (2.4, p.55)

enum class: **enum class**는 독립된 **타입**이며 열거자의 스코프를 해당 **enum class**로 제한한다. [U02-002] (2.5, p.58)

원문의 데이터 표현 구분

구분	책의 설명
struct [U02-084] (2.3, p.55)	**struct**는 멤버가 기본적으로 public인 **class**이며, **struct**에도 **생성자**를 포함한 멤버 함수를 정의할 수 있다. [U02-077] (2.3, p.55)
variant [U02-085] (2.4, p.57)	**variant**는 대안적인 여러 **타입**의 값 중 하나를 제공하며, 대부분의 상황에서 공용체를 대체할 수 있다. [U02-079] (2.4, p.57)

제3장. 모듈화

핵심 정의

네임스페이스: **네임스페이스**는 일부 선언을 함께 묶고 다른 이름과 충돌하지 않게 하는 메커니즘이다. [U03-001] (3.4, p.68)

사전 조건: **사전 조건**은 함수 작동의 기본적인 가정이다. [U03-151] (3.5.2, p.73)

클래스 불변 조건: 클래스 수준에서 보장돼야 할 조건을 **클래스 불변 조건**이라고 한다. [U03-152] (3.5.2, p.74)

에러 확인 시점의 구분

구분	책의 설명
예외 [U03-159] (3.5.5, p.80)	예외는 실행 시간에 발생하는 에러를 보고하는 용도로 사용한다. [U03-160] (3.5.5, p.80)
st**at**ic_assert [U03-161] (3.5.5, p.80)	st**at**ic_assert(A,S)는 A가 true가 아니면 S를 컴파일러 에러 메시지로 출력한다. [U03-158] (3.5.5, p.80)

제4장. 클래스

핵심 정의

구체 타입: **구체 타입**은 **타입** 정의의 일부로 메모리 표현을 포함하며, 내장 **타입**처럼 작동하는 것이 기본 아이디어다. [U04-001] (4.2, p.93)

추상 타입: **추상 타입**은 사용자에게 상세 구현을 완전히 감추며, 인터페이스와 메모리 표현을 분리한다. [U04-153] (4.3, p.102)

컨테이너: **컨테이너**는 요소의 컬렉션을 저장하는 **객체**다. [U04-151] (4.2.2, p.97)

상속의 두 역할

구분	책의 설명
인터페이스 상속 [U04-162] (4.5.1, p.112)	**인터페이스 상속**은 기반 클래스 **객체**를 사용하는 곳에서 파생 클래스 **객체**를 대신 사용할 수 있게 한다. [U04-155] (4.5.1, p.112)
구현 상속 [U04-163] (4.5.1, p.112)	**구현 상속**은 기반 클래스의 함수와 데이터 멤버로 파생 클래스의 구현을 단순하게 한다. [U04-002] (4.5.1, p.112)

제5장. 필수적인 연산

핵심 정의

Vector 복사 생성자: 원문의 **Vector 복사 생성자**는 새 저장 공간을 할당하고 요소를 복사하여 각 Vector가 자기 요소의 복사본을 갖게 한다. [U05-139] (5.2.1, p.126)

Vector 이동 생성자: 원문의 **Vector 이동 생성자**는 원본의 elem과 sz를 가져오고 원본을 nullptr와 크기 0으로 만든다. [U05-141] (5.2.2, p.129)

std::move: move()는 아무것도 이동시키지 않고 앞으로 이동시킬 인자의 **rvalue 참조**를 반환하는 일종의 **타입** 변환이다. [U05-002] (5.2.2, p.130)

원문 Vector의 복사와 이동

구분	책의 설명
Vector 복사 [U05-148] (5.2.1, p.126)	원문의 **Vector 복사 생성자**는 새 저장 공간을 할당하고 요소를 복사하여 각 Vector가 자기 요소의 복사본을 갖게 한다. [U05-139] (5.2.1, p.126)
Vector 이동 [U05-149] (5.2.2, p.129)	원문의 **Vector 이동 생성자**는 원본의 elem과 sz를 가져오고 원본을 nullptr와 크기 0으로 만든다. [U05-141] (5.2.2, p.129)

제6장. 템플릿

핵심 정의

템플릿: **템플릿**은 **타입**이나 값의 집합을 파라미터화한 클래스나 함수다. [U06-001] (6.1, p.141)

함수 객체: **함수 객체**는 함수처럼 호출할 수 있는 **객체**다. [U06-109] (6.3.2, p.151)

술어: **술어**는 호출 가능하며 그 결과로 true나 false를 반환하는 무언가를 말한다. [U06-110] (6.3.2, p.152)

파라미터화된 연산의 표현

구분	책의 설명
함수 객체 [U06-118] (6.3.2, p.151)	**함수 객체**는 함수처럼 호출할 수 있는 **객체**다. [U06-109] (6.3.2, p.151)
람다 표현식 [U06-119] (6.3.3, p.153)	**람다 표현식**은 **함수 객체**를 암묵적으로 만들어내는 표기법이다. [U06-111] (6.3.3, p.153)

제7장. 컨셉과 제네릭 프로그래밍

핵심 정의

컨셉: **컨셉**은 하나 이상의 **타입**을 어떻게 사용할 수 있는지 지정하는 컴파일 시간 **술어**다. [U07-001] (7.2.4, p.170)

리프팅: **리프팅**은 동일한 성능을 유지하면서 구체적인 코드를 일반화하는 과정이다. [U07-105] (7.3.2, p.175)

파라미터 팩: ...으로 선언된 파라미터를 **파라미터 팩**이라고 한다. [U07-106] (7.4, p.176)

원문이 설명한 타입 검사 시점

구분	책의 설명
컨셉 사용 [U07-112] (7.5, p.181)	**컨셉**을 사용하면 먼저 **템플릿** 인자의 **컨셉** 만족 여부를 검사하고, 그 시점에 확인할 수 없는 것은 인스턴스화 시간까지 미룬다. [U07-111] (7.5, p.181)
컨셉 미사용 [U07-113] (7.5, p.181)	**컨셉**을 사용하기 이전 코드에서는 모든 **타입** 검사를 인스턴스화 시간에 수행한다. [U07-114] (7.5, p.181)

제8장. 라이브러리의 개요

핵심 정의

표준 라이브러리: C++ **표준 라이브러리**는 가장 널리 사용하는 기본 자료 구조와 그 자료 구조에서 동작하는 알고리즘을 제공한다. [U08-001] (8.2, p.186)

STL: **컨테이너**와 **find**(), sort(), merge() 등을 비롯한 알고리즘 프레임워크를 관례적으로 **STL**이라고 한다. [U08-040] (8.2, p.185)

표준 헤더: **표준 라이브러리**의 모든 기능은 **표준 헤더**에서 제공된다. [U08-041] (8.3, p.186)

원문 헤더와 제공 기능

구분	책의 설명
<**string**> [U08-045] (8.3, p.186)	<**string**> 헤더를 이용하면 표준 **string**을 사용할 수 있다. [U08-046] (8.3, p.186)
<**list**> [U08-047] (8.3, p.186)	<**list**> 헤더를 이용하면 표준 **list**를 사용할 수 있다. [U08-048] (8.3, p.186)

제9장. 문자열과 정규 표현식

핵심 정의

string: **string**은 문자 시퀀스를 저장하고 조작하는 **Regular 타입**이다. [U09-001] (9.2, p.190,192)

string_view: **string_view**는 연속된 문자 시퀀스를 나타내는 포인터와 길이의 쌍으로 볼 수 있으며, 문자를 소유하지 않는다. [U09-121] (9.3, p.194-195)

정규 표현식: **정규 표현식**은 텍스트 안의 패턴을 간단히 기술하고 찾는 도구다. [U09-038] (9.4, p.197-198)

원문에서 설명한 역할·특징

항목	책의 설명
string [U09-122] (9.2, p.190,192)	**string**은 문자 시퀀스를 저장하고 조작하는 **Regular 타입**이다. [U09-001] (9.2, p.190,192)
string_view [U09-123] (9.3, p.194-195)	**string_view**는 연속된 문자 시퀀스를 나타내는 포인터와 길이의 쌍으로 볼 수 있으며, 문자를 소유하지 않는다. [U09-121] (9.3, p.194-195)

제10장. 입력과 출력

핵심 정의

입출력 스트림: **입출력 스트림** 라이브러리는 텍스트와 수치 값의 형식화·비형식화 및 버퍼링된 입출력을 지원한다. [U10-001] (10.1, p.210-211)

cout·cerr: cout은 표준 출력, cerr은 에러 보고 스트림이다. [U10-002] (10.2, p.211,213)

cin: **cin**은 표준 입력 스트림이며, >>는 istream에서 값을 가져오는 연산이다. [U10-143] (10.3, p.213,215)

원문에서 설명한 역할·특징

항목	책의 설명
파일 스트림 [U10-147] (10.7, p.221-222)	<fstream>의 ifstream은 읽기, ofstream은 쓰기, fstream은 읽기와 쓰기용이다. [U10-061] (10.7, p.221-222)
문자열 스트림 [U10-148] (10.8, p.222-223)	<sstream>의 i**string**stream은 **string**에서 읽고 o**string**stream은 **string**에 쓰며 **string**stream은 둘 다 한다. [U10-068] (10.8, p.222-223)

제11장. 컨테이너

핵심 정의

컨테이너: **컨테이너**는 **객체**를 저장하는 일이 주목적인 클래스다. [U11-001] (11.1, p.234)

vector<T>: **vector<T>**는 T **타입** 요소가 메모리에 연속적으로 저장되는 시퀀스다. [U11-002] (11.2, p.234-235)

list: **list**는 이중 연결 리스트이며, 다른 요소를 이동시키지 않고 시퀀스 중간에 삽입·삭제할 때 사용한다. [U11-134] (11.3, p.241,243)

원문에서 설명한 역할·특징

항목	책의 설명
map [U11-140] (11.4, p.244-245)	**map**은 키와 값의 쌍을 담고 조회에 최적화된 연관 **컨테이너**다. [U11-055] (11.4, p.244-245)
unordered_map [U11-141] (11.5, p.246,248)	**unordered_map**은 해시 테이블을 이용하며 요소 사이의 순서를 유지하지 않는다. [U11-135] (11.5, p.246,248)

제12장. 알고리즘

핵심 정의

반개구간: 입력 시퀀스는 첫 요소를 포함하고 끝 위치는 포함하지 않는 **반개구간 [begin:end)**이며, 첫 요소와 마지막 요소 다음을 가리키는 **반복자** 쌍으로 나타낸다. [U12-001] (12.1, p.254-255)

반복자: **반복자**는 특정 **타입**의 **객체**이며 저장 구조에 맞는 정보를 가진다. [U12-002] (12.3, p.259-260)

술어: **술어**는 알고리즘이 각 요소에 적용할 조건을 전달하는 수단이다. [U12-035] (12.5, p.263-264)

원문에서 설명한 역할·특징

항목	책의 설명
find [U12-163] (12.2, p.255-256)	**find**는 찾은 요소의 **반복자**를 반환하고, 찾지 못하면 입력 구간의 end()를 반환한다. [U12-010] (12.2, p.255-256)
find_if [U12-164] (12.5, p.263-264)	**find_if**는 전달된 **술어**가 참이 되는 첫 요소를 찾는다. [U12-161] (12.5, p.263-264)

제13장. 유틸리티

핵심 정의

자원: **자원**은 획득한 후 언젠가 해제해야 하는 것으로 메모리·소켓·스레드 **핸들**·파일 **핸들**이 예다. [U13-001] (13.2, p.276-277)

unique_ptr·shared_ptr: **unique_ptr**은 유일한 소유권을, **shared_ptr**은 공유된 소유권을 표현한다. [U13-002] (13.2.1, p.277,281)

gsl::span: **gsl::span**은 요소 시퀀스의 포인터와 길이를 함께 표현하는 구간 참조이며, 그 요소를 소유하지 않는다. [U13-191] (13.3, p.284,286)

원문에서 설명한 역할·특징

항목	책의 설명
optional<A> [U13-196] (13.5.2, p.298-299)	**optional<A>**는 A 또는 아무것도 없음을 나타낸다. [U13-096] (13.5.2, p.298-299)
any [U13-197] (13.5.3, p.299-300)	**any**는 임의의 **타입**을 저장하며 저장된 **객체**가 있으면 그 **타입**을 기억한다. [U13-102] (13.5.3, p.299-300)

제14장. 수치

핵심 정의

수치 알고리즘: <numeric>은 시퀀스의 합·내적·부분합·이웃한 차이 등을 일반화한다. [U14-001] (14.3, p.318-319)

reduce: **reduce**는 accumul**at**e와 달리 요소 연산의 순서가 정해지지 않는다. [U14-002] (14.3.1, p.319-320)

complex<Scalar>: <complex>의 **complex<Scalar>**는 실수부와 허수부 **타입**을 매개**변수**로 받는 복소수 **템플릿**이다. [U14-053] (14.4, p.320-321)

원문에서 설명한 역할·특징

항목	책의 설명
reduce [U14-104] (14.3.1, p.319-320)	**reduce**는 accumul**at**e와 달리 요소 연산의 순서가 정해지지 않는다. [U14-002] (14.3.1, p.319-320)
valarray [U14-105] (14.6, p.323-324)	<**valarray**>의 **valarray**는 일반성이 다소 낮은 대신 수치 계산 최적화에 더 적합하도록 제공되는 배열 **템플릿**이다. [U14-066] (14.6, p.323-324)

제15장. 동시성

핵심 정의

동시성: **동시성**은 여러 **태스크**를 동시에 실행하는 것이다. [U15-001] (15.1, p.327-328)

태스크·thread: **태스크**는 다른 계산과 동시에 실행할 수 있는 작업이며, **thread**는 이를 표현하는 시스템 수준 개념이다. [U15-002] (15.2, p.328-329)

mutex: **mutex**는 상호 배제 **객체**이며 lock()으로 획득하고 unlock()으로 해제한다. [U15-108] (15.5, p.333,335)

원문에서 설명한 역할·특징

항목	책의 설명
future·promise [U15-116] (15.7.1, p.339-340)	**promise**에 값을 넣으면 연결된 **future**에서 그 값을 볼 수 있으며, **future**<X>::get()은 필요한 경우 계산이 끝날 때까지 기다린다. [U15-110] (15.7.1, p.339-340)
packaged_task [U15-117] (15.7.2, p.340-341)	**packaged_task**는 **태스크**의 반환값이나 예외를 **promise**에 넣는 래퍼를 제공하고, get_**future**()로 대응 **future**를 돌려준다. [U15-069] (15.7.2, p.340-341)

제16장. 역사와 호환성

핵심 정의

WG21: ISO C++ 위원회는 **WG21**로 불린다. [U16-001] (16.1.3, p.355,357)

C++11: 두 번째 표준 **C++11**은 2011년 8월 21:0으로 승인됐다. [U16-002] (16.1.3, p.354-355)

표준과 스타일: 표준은 기능의 동작을 규정하지만 효과적인 사용법을 모두 알려주지는 않는다. [U16-065] (16.1.4, p.357-358)

원문에서 설명한 역할·특징

항목	책의 설명
표준과 스타일 [U16-196] (16.1.4, p.357-358)	표준은 기능의 동작을 규정하지만 효과적인 사용법을 모두 알려주지는 않는다. [U16-065] (16.1.4, p.357-358)
지원 중단 [U16-197] (16.2.7, p.364-365)	**지원 중단**은 해당 기능을 피하라는 강한 신호이지만 즉시 제거와 동일하지 않다. [U16-128] (16.2.7, p.364-365)